ZemiFix Architecture Specification v1

Nov 4, 2026 · @Erasmus Mensah-Ackon

ZemiFix is a web and mobile maintenance management system (CMMS) that lets any organization plan, assign, track and cost its maintenance work, and lets service contractors run repair jobs for paying clients and bill them from the same system.

Purpose of this document: the single reference for the tech lead and for AI coding tools that scaffold and build ZemiFix module by module. It replaces all earlier drafts.

1. Product definition

1.1 Two operating modes, one codebase

A company turns on one or both modes in settings. Every feature below belongs to a mode or to the shared core.

Mode	Who uses it	What it does
Facility mode (priority 1)	Hotels, restaurants, clinics, hospitals, schools, factories, estates	Registers the company's own assets, schedules preventive maintenance, handles internal requests, tracks downtime and cost per asset
Contractor mode (priority 2)	Maintenance and repair companies working for customers	Adds customers, sites, contracts and SLAs, quotes, billable jobs, invoices, statements and a customer portal
Shared core	Both	Work requests, work orders, assets, inventory, purchasing, mobile app, notifications, reports

1.2 Personas

Persona	Main goal	Main surface
Requester (hotel guest services, ward nurse, restaurant manager)	Report a fault in under 30 seconds and see progress	Mobile web, QR scan, email or WhatsApp link

Persona	Main goal	Main surface
Technician	See today's jobs, record work, parts and time, close the job	Mobile PWA, offline capable
Planner or dispatcher	Turn requests into scheduled, assigned work	Web app, calendar and board
Supervisor or manager	Approve, balance workload, watch SLA and backlog	Web dashboard
Storekeeper	Keep stock accurate, issue and receive parts	Web and mobile scanner
Finance	Cost jobs, invoice customers, chase payments	Web app
Customer contact (contractor mode)	Raise requests, see progress, approve quotes, pay, download statements	Customer portal
Executive	See cost, uptime and performance without asking anyone	Dashboards and scheduled reports
Admin	Configure the system and users	Web app

1.3 Structural decisions (fixed)

- **Single company.** One legal entity per deployment. One `Company` record holds name, currency, timezone, tax scheme and settings. There is no multi-company or tenant layer.
- **Branch tree.** Branches nest to any depth (Head Office, Region, Branch, Outlet). Each branch has a manager, an address, and optionally its own invoice prefix and store.
- **Department tree.** Departments nest (Operations, Engineering, Electrical). They are company-wide: the same tree applies at every branch, and a record carries both a branch and a department.
- **Placement of users.** Every user belongs to one branch and one department and holds an **access scope** (Own, Department, Branch, Branch and below, Company). The role decides what the user may do; the scope decides how far the user can see and act.
- **Teams** are flat skill groups (Plumbing, HVAC, Biomedical) used for assignment and workload. They carry no visibility rules.

- **Locations are physical.** A site, building, floor and room tree describes where an asset is. It is separate from the branch tree.
- **Dual control wherever money or stock moves.** No transaction takes effect until a second person approves it, and nobody can approve what they keyed (section 5.6).
- **Mandatory OTP login** for every staff user and every portal contact (section 9.1).
- **Full activity audit trail** of every user action, including views of sensitive records (section 9.6).
- **International by configuration.** Currency, timezone, language, date format and tax scheme are company settings. Payment, SMS and email providers are pluggable.
- **Stack:** Next.js (TypeScript), PostgreSQL, Prisma.

2. Competitive feature map

A CMMS is competitive when it covers the category baseline completely and is better than the field on a few things buyers feel daily. This table is the scope checklist: each row is a capability area, what must exist, and where ZemiFix goes further. Release tags (R1 to R3) are explained in section 10.

Area	Category baseline (must have)	ZemiFix advantage	Release
Requests	Web form, status tracking, approval	QR code on every asset and room, no login needed for a fault report, WhatsApp and SMS intake, photo and voice note, duplicate detection	R1, R2
Work orders	Lifecycle, assignment, tasks, labor, parts, comments, attachments	Checklists with pass/fail, required photos and signatures, sub-tasks, linked orders, one-tap reopen with reason	R1
Preventive maintenance	Time-based and meter-based schedules, auto work orders	Condition triggers from readings, seasonal schedules, schedule calendar with load view, bulk assignment of asset to template	R1, R2

Area	Category baseline (must have)	ZemiFix advantage	Release
Assets	Hierarchy, meters, downtime log, warranty	Bill of materials per asset, document library, lifecycle cost, replace-or-repair advice from cost history, asset import from spreadsheet	R1, R2
Mobile	Mobile access to assigned jobs	Installable PWA that works offline, sync with conflict handling, barcode and QR scan, camera upload, low-bandwidth mode	R1
Inventory	Parts, stock levels, reorder points	Multi-store with bins, issue against work order, cycle counts, kits per PM template, serial and batch tracking	R1, R2
Purchasing	Vendors, purchase orders, receiving	Auto reorder suggestions, approval limits, goods receipt with invoice match, vendor performance score	R2
SLA and contracts	Response and resolution targets	Business-hours calendars, pause on customer-hold, escalation ladder, breach forecast before it happens	R1, R2
Customers (contractor mode)	Customer records, jobs per customer	Sites, contracts, rate cards, quotes with approval link, recurring service agreements, customer portal with statements	R1, R2
Billing	Invoice from work orders	Quote to job to invoice chain with maker-checker approval before issue, labor and parts markup rules, tax lines, credit notes, payment links, aged debtors, statements	R1, R2

Area	Category baseline (must have)	ZemiFix advantage	Release
Reporting	Standard reports, CSV export	Live dashboards, MTTR, MTBF, PM compliance, backlog age, technician utilization, cost per asset, scheduled email reports, saved views	R1, R2
Notifications	Email and SMS	Requester told at every status change by email, per-event rules, quiet hours, WhatsApp, digest mode, in-app inbox	R1, R2
Security	Roles, passwords	Mandatory OTP login, granular permissions with separate maker and checker rights, dual control on every transaction, full activity audit trail, branch and department scoping, signed file URLs	R1
Integration	Import and export	Public REST API, webhooks, accounting export, calendar feeds, SSO	R2, R3
Intelligence	Rarely offered well	Failure-code analysis, predictive PM suggestions, AI triage of requests (category, priority, likely parts), natural-language search	R3

Evidence status. The baseline column reflects capabilities commonly offered across the CMMS category, written from general product knowledge. It has not been checked against named competitors. A first task in R1 is a dated, sourced comparison of three to five competitors in the target markets, used to confirm which rows are table stakes and which are real differentiators.

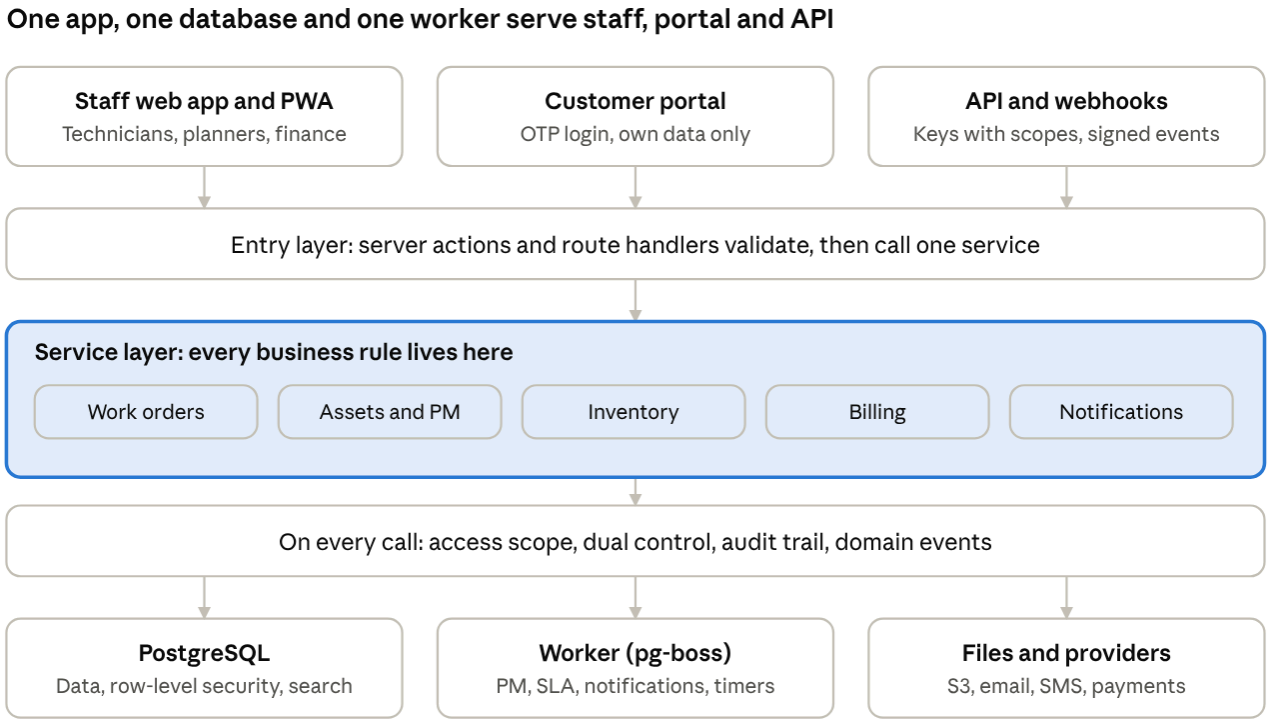
Where ZemiFix competes hardest

1. **Speed to first value.** A hotel or clinic can import assets from a spreadsheet, print QR labels and have technicians on the PWA in one day.

2. **One system for both worlds.** Most tools serve either in-house teams or service contractors. ZemiFix serves both with one data model.
3. **Works on poor connections.** Offline-first technician app and light pages suit sites with unreliable networks.
4. **Affordable to run.** Single Postgres, no Redis or search cluster, deployable on one VPS.

3. Architecture

ZemiFix is a modular monolith: one Next.js codebase, one PostgreSQL database and one background worker process. This is the cheapest shape to build, hire for and run, and the modules inside it can be split out later if a real need appears.



architecture · 5 layers, 3 entry points

Every entry point reaches the data only through the service layer, where scope, dual control and audit are applied once.

3.1 Stack

Concern	Choice	Reason
Framework	Next.js (App Router), TypeScript strict	One codebase for staff app, portal, PWA and API

Concern	Choice	Reason
UI	React, Tailwind CSS, shadcn/ui, TanStack Table	Fast to build, accessible, themeable
Database	PostgreSQL 16	Relational integrity, JSONB for custom fields, full-text search, row locking
ORM	Prisma	Typed queries, migrations, easy for new hires
Validation	Zod, shared by forms, actions and API	One schema, no drift
Staff auth	Auth.js: password, then mandatory OTP by email or SMS; optional SSO (OIDC) later	Standard, replaceable
Portal auth	Separate realm: email or phone one-time code	Customers are never staff users
Background jobs	pg-boss (queue and scheduler on Postgres)	No Redis to run
Files	S3-compatible storage, signed URLs	Not on app server disk
Search	Postgres full-text plus pg_trgm	No search cluster
Realtime	Server-sent events fed by Postgres LISTEN/NOTIFY	Live boards without a socket server
Offline	Service worker, IndexedDB outbox, background sync	Technician app on weak networks
Email, SMS, WhatsApp	Provider interfaces with one adapter each	Swap providers per country
Payments	Provider interface (card, mobile money, bank link)	Stripe, Paystack or Flutterwave adapters
Observability	pino logs, OpenTelemetry traces, Sentry errors	Find faults before customers do
Tests	Vitest, Playwright, Testcontainers (real Postgres)	Business rules tested against the real database

3.2 Layers and rules

1. **UI layer.** React components and pages. No business rules.
2. **Entry layer.** Server actions and route handlers. They authenticate, validate with Zod, call exactly one service method, and return the result.
3. **Service layer** (`src/modules/<domain>/service.ts`). All business rules, permission checks, transactions, audit entries and event publishing live here. This is the only layer allowed to write.
4. **Data layer.** Prisma access inside services. Raw SQL only for reporting views and locking queries.
5. **Event bus (in process).** Services publish domain events (`workOrder.completed`). Notification, SLA, audit-feed and webhook handlers subscribe. Handlers enqueue jobs, so a failing email never fails a work order.

Cross-cutting rules every service method follows:

- Resolve the acting user from the session, never from a request field.
- Check the permission, then the scope (branch and department), before touching data.
- Apply scope through one mechanism: a Prisma client extension that adds the branch and department filter to every query on scoped models. Raw SQL for reports uses the same scope helper.
- Add PostgreSQL Row-Level Security on branch-scoped tables as a database-layer backstop, driven by a session variable set per request. A bug in the application layer then cannot leak another branch's rows.
- Route every controlled transaction through the dual-control engine (section 5.6). A service method creates it in `PendingApproval`; it takes effect only when a different, authorized user approves it.
- Wrap multi-row writes in one transaction.
- Write an audit record for every create, update, status change, delete, approval, login, export and sensitive view (section 9.6).
- Use optimistic concurrency: the client sends `version`, a mismatch returns HTTP 409.
- Soft delete only. Nothing is hard deleted except by a retention job.

3.3 Repository layout

```
src/
  app/            # routes: (staff), (portal), (public), api/
  modules/        # one folder per domain: service, schema, events, policies
    assets/ work-requests/ work-orders/ pm/ inventory/ purchasing/
    customers/ contracts/ quotes/ invoicing/ notifications/ reports/
    identity/ settings/ audit/ integrations/
  platform/       # db, auth, queue, storage, mailer, sms, payments, search,
sse
  worker/         # job handlers and schedules (separate process)
  pwa/            # service worker, offline outbox, sync
prisma/           # schema.prisma, migrations, seed
tests/            # unit, integration, e2e
```

3.4 Deployment

- **Default:** Docker Compose on one VPS with three containers (web, worker, Postgres), Caddy for HTTPS, and S3-compatible storage off the server.
- **Managed option:** web on a Node host, managed Postgres, worker as a second service. The queue lives in Postgres so nothing else is needed.
- **Backups:** nightly logical dump plus continuous WAL archive where available, copied off-server, with a monthly restore test.
- **Environments:** local, staging, production, each with its own database. Migrations run in the release pipeline, never by hand.

3.5 Background work

Job	Schedule	Purpose
Generate PM work orders	Daily, early morning (org timezone)	Create due orders idempotently
SLA clock check	Every 5 minutes	Flag at-risk and breached orders, escalate
Notification dispatch	Queue, continuous	Send email, SMS, WhatsApp with retry and back-off
Overdue invoices	Daily	Mark overdue, send reminders
Low stock and reorder	Daily	Alert and draft purchase orders
Stock ledger reconcile	Nightly	Assert on-hand equals ledger sum

Job	Schedule	Purpose
Report snapshots	Nightly	Pre-compute KPIs for dashboards
Scheduled reports	Per user setting	Email PDF or CSV
Retention and backup checks	Weekly	Purge expired files, verify backup freshness

Further scheduled jobs added by the control features: approval reminders and escalation (every 15 minutes), audit hash anchoring (daily), audit archive to cold storage (monthly), and OTP and notification delivery health check (hourly).

3.6 Operating many deployments

Because each customer runs its own single-company deployment, the product team needs fleet operations from day one.

Concern	Approach
Provisioning	One script creates a deployment: infrastructure as code, database, storage bucket, subdomain, TLS, seed data (roles, permissions, approval policies, templates) and the first Admin invitation. Target: under 30 minutes, no manual steps
Configuration	One versioned configuration file per deployment (modes, branding, limits, providers); no code forks per customer
Releases	One release train. New versions roll out in rings: internal deployment, pilot customers, then everyone. Migrations must be backward compatible for one version so a rollback is safe
Monitoring	A central view of every deployment: uptime, error rate, job heartbeat, backup freshness, disk and database size
Backups	Per deployment, encrypted, restore-tested on a schedule
Support access	No standing access to customer data. Support access is requested, approved by the customer's Admin, time-limited and fully audited

Concern	Approach
Licensing and usage	A licence record per deployment (users, features, expiry) and usage counters (active users, assets, SMS sent) for billing and capacity planning
Exit	Full data export in open formats on request

4. Data model

Field lists, not migration code. Conventions are stated once and apply to every table. Records are placed in the company through `branchId` and `departmentId`; there is no tenant or company id because there is one company per deployment.

4.1 Global conventions

Field	Rule
`id`	UUID v7 primary key (time-ordered, safe in URLs and sync)
`number`	Human reference on documents (`WO-000417`), unique, from a gap-tolerant sequence with configurable prefix; invoices use a gapless sequence per branch prefix
`branchId`	Required and indexed on branch-scoped tables (assets, requests, orders, stores, invoices, purchase orders, payments, users). Filtered by the user's access scope
`departmentId`	Nullable and indexed on requests, orders, assets, requisitions, purchase orders. A null department is visible to the whole branch so unassigned work can be triaged
`createdAt` , `updatedAt`	Set by the data layer
`createdById` , `updatedById`	Set by the service from the session, never by the caller. `createdById` is the maker for dual control
`approvalState`	On controlled transactions: Draft, PendingApproval, Approved, Rejected. Only Approved records affect stock, balances and invoices

Field	Rule
`deletedAt`	Soft delete on every business table
`version`	Integer for optimistic concurrency, incremented in the same transaction
`customFields`	JSONB on assets, requests, work orders, customers, parts, validated against admin-defined field definitions
Money	`Decimal(14,2)` plus currency code on documents; never floats
Time	Stored in UTC, displayed in the company timezone

4.2 Identity and settings

```
Company             single row: name, legalName, logo, baseCurrency,
timezone, locale,
                    dateFormat, taxScheme, fiscalYearStart,
modes{facility,contractor},
                    numberPrefixes, businessHoursCalendarId, otpChannel,
sessionPolicy
Branch              parentId (self), name, code (unique),
type(HeadOffice|Region|Branch|Outlet),
                    address, phone, managerUserId?, alternateApproverId?,
invoicePrefix?, isActive,
                    path (materialized, e.g. /1/4/9/)     // fast "branch and
below" filters
Department          parentId (self), name, code (unique), headUserId?,
isActive, path
User                name, email (unique), phone, passwordHash (argon2id),
roleId,
                    branchId, departmentId,
accessScope(Own|Department|Branch|BranchAndBelow|Company),
                    teamIds[], approvalLimit?, isActive, locale, lastLoginAt,
version
UserScopeGrant      userId, branchId?, departmentId?, validFrom, validTo
(extra reach, time-boxed)
Role                name, isSystem, description
Permission          resource, action                      // maker and
checker are separate actions:
                                                          // invoice.create
vs invoice.approve
RolePermission      roleId + permissionId
```

```
Team                name, description, leadUserId         (flat skill group)
Session             userId, expiresAt, ip, userAgent, deviceId, otpVerifiedAt
(revocable)
OtpCode             userId or contactId,
purpose(Login|StepUp|PasswordReset|PhoneVerify|PortalLogin),
                    channel(Email|SMS), codeHash, expiresAt, attempts,
usedAt, ip
TrustedDevice       userId, deviceHash, trustedUntil      (optional, admin
can disable)
ApiKey              name, hash, scopes[], lastUsedAt, expiresAt
BusinessCalendar    name, weekly hours, holidays[]        (drives SLA clocks)
CustomFieldDef      entity, key, label, type, options, isRequired
NumberSequence      entity, branchId?, prefix, next, isGapless
```

4.3 Locations and assets

```
Location            parentId (self), name, branchId,
type(Site|Building|Floor|Room|Area), code, qrToken
AssetCategory       name, parentId, defaultChecklistTemplateId,
defaultPmTemplateIds[]
Asset               number, name, tag (unique), branchId, departmentId?,
categoryId, locationId, parentAssetId (self),
                    criticality(Low|Medium|High|Critical),
status(Operational|Degraded|Down|
                    UnderMaintenance|Standby|Retired), make, model,
serialNumber,
                    purchaseDate, purchaseCost, warrantyExpiry,
expectedLifeMonths,
                    vendorId, ownerCustomerId, customerSiteId, qrToken,
customFields, version
                    // ownerCustomerId null = organization's own asset
(facility mode)
                    // ownerCustomerId set  = customer-owned asset serviced
under contract
AssetMeter          assetId, type, unit, currentReading, lastReadingAt
MeterReading        meterId, value, readAt, source(Manual|Import|Sensor),
recordedById
AssetBom            assetId, partId, quantity       (parts that belong to
this asset)
AssetDocument       assetId, fileId, type(Manual|Warranty|Certificate|Photo),
expiresAt
DowntimeLog         assetId, workOrderId?, start, end?, reason,
impact(Partial|Full)
AssetStatusHistory  assetId, from, to, changedById, at, note
```

4.4 Work requests and work orders

```
WorkRequest          number, source(Web|QR|Portal|Email|WhatsApp|Phone|Walk-
in),
                     title, description, priority(Low|Medium|High|Urgent),
category,
                     branchId, departmentId?, locationId?, assetId?,
customerId?, customerSiteId?,
                     requesterUserId? | requesterContactId? |
requesterName+phone+email (guest),

status(Submitted|UnderReview|Approved|Rejected|Converted|Cancelled),
                     reviewedById, reviewedAt, rejectionReason,
duplicateOfId?, version
WorkOrder            number,
type(Corrective|Preventive|Inspection|Installation|Emergency|Project),
                     title, description, priority, branchId, departmentId?,
assetId?, locationId, requestId?,
                     customerId?, contractId?,
jobKind(Warranty|Contract|Billable|Internal),

status(Draft|PendingApproval|Open|Assigned|InProgress|OnHold|Completed|Verifi
ed|Closed|Cancelled),
                     holdReason?, teamId?, assignedToId?, plannedStart,
plannedEnd,
                     actualStart, actualEnd, estimatedHours, estimatedCost,
actualCost,
                     failureCodeId?, causeCodeId?, remedyCodeId?,
                     slaResponseDueAt, slaResolutionDueAt, slaRespondedAt,
slaPausedMs, slaBreached,
                     requiresVerification, verifiedById?,
customerSignatureFileId?, version
WorkOrderStatusHistory  workOrderId, from, to, changedById, at, note   (same
transaction)
WorkOrderTask        workOrderId, sequence, description,
type(Check|Measure|Text|Photo),
                     expectedValue?, result?, isCompleted, completedById,
completedAt
WorkOrderLabor       workOrderId, userId, startedAt, endedAt, hours, rate,
billableRate, notes
WorkOrderPart        workOrderId, partId, quantity, unitCost, billablePrice,
storeId
WorkOrderExpense     workOrderId, description, amount, receiptFileId,
isBillable   (travel, hire)
WorkOrderComment     workOrderId, authorUserId? | authorContactId?, body,
isVisibleToCustomer
```

```
WorkOrderLink       fromId, toId, type(Blocks|Follows|Duplicate|Related)
Attachment          ownerType, ownerId, fileId, caption,
scanStatus(Pending|Clean|Rejected)
FailureCode         type(Problem|Cause|Remedy), code, label, parentId
```

4.5 Preventive maintenance

```
ChecklistTemplate   name, description, categoryId?, estimatedHours,
requiredSkills[]
ChecklistItem       templateId, sequence, description, type, expectedValue,
isRequired
PmPlan              name, assetId | categoryId (bulk), checklistTemplateId,
teamId?, assignedToId?,
                    trigger(Time|Meter|Condition|Seasonal), intervalValue,
intervalUnit,
                    meterId?, meterThreshold?, conditionRule? (JSON),
leadTimeDays,
                    startDate, endDate?,
anchor(FixedCalendar|FromCompletion), isActive
PmOccurrence        planId, dueDate, workOrderId?,
status(Planned|Generated|Skipped|Missed)
                    UNIQUE (planId, dueDate)   // idempotent generation
PmPlanPartKit       planId, partId, quantity    (parts reserved when order is
generated)
```

Checklist items are copied into `WorkOrderTask` at generation time. Editing a template later never changes past orders.

4.6 Inventory and purchasing

```
Store               name, branchId, locationId, isActive            (several
stores allowed, no hierarchy)
Part                sku (unique), name, description, categoryId,
unitOfMeasure, barcode,
                    isSerialised, isBatchTracked, reorderPoint,
reorderQuantity,
                    standardCost, preferredVendorId, customFields, version
Stock               partId, storeId, bin, quantityOnHand, quantityReserved
UNIQUE (partId, storeId, bin)
StockTransaction    partId, storeId, bin,
type(Receipt|Issue|Return|Transfer|Adjustment|CountCorrection),
                    quantity (signed), unitCost, workOrderId?,
purchaseOrderId?, transferId?,
                    performedById, branchId, at, reference, approvalId?
// the ledger: append only
StockCount          storeId, status, startedAt, closedAt;  StockCountLine
countId, partId, bin, expected, counted
PartRequest         number, branchId, departmentId?, workOrderId?,
requestedById, storeId, status(Requested|Approved|
                    PartiallyIssued|Issued|Rejected|Cancelled), approvedById,
approvedAt;  PartRequestLine
Vendor              name, contact, email, phone, address, taxNumber,
category, rating, paymentTerms, isActive
PurchaseRequisition number, branchId, departmentId?, requestedById,
status(Draft|PendingApproval|Approved|Rejected), approvedById;  lines
PurchaseOrder       number, vendorId, branchId, departmentId?,
status(Draft|PendingApproval|Approved|Sent|PartiallyReceived|Received|
                    Cancelled), orderDate, expectedDate, currency, subtotal,
tax, total, version
PurchaseOrderLine   poId, partId, quantity, quantityReceived, unitCost
GoodsReceipt        poId, receivedById, receivedAt, deliveryNoteNo;  lines
VendorInvoice       poId, vendorId, number, amount, status(Open|Matched|Paid)
(three-way match)
```

4.7 Customers, contracts, quotes and billing (contractor mode)

```
Customer            number, name, type(Individual|Business), email, phone,
billingAddress, taxNumber,
                    paymentTerms, creditLimit?, priceListId?,
accountManagerId?, primaryBranchId?, isActive
CustomerSite        customerId, name, address, geo, contactPhone, accessNotes
CustomerContact     customerId, name, email, phone, position, portalEnabled,
notificationPrefs
Contract            number, customerId,
```

```
type(Maintenance|CallOut|Project|Warranty), startDate, endDate,
                    slaPolicyId, businessCalendarId,
billingMode(Fixed|TimeAndMaterial|Retainer|PerVisit),
                    fixedAmount?, billingFrequency, rateCardId,
includedHours?, autoRenew, status
ContractAsset       contractId, assetId             (covered equipment)
SlaPolicy           name; SlaRule  policyId, priority, responseMinutes,
resolutionMinutes,
                    escalationSteps[]
RateCard            name; RateCardLine  cardId, role|skill, hourlyRate,
callOutFee, overtimeMultiplier
PriceList           name, currency; PriceListLine  listId, partId, price |
markupPercent
Quote               number, customerId, siteId,
status(Draft|PendingApproval|Released|Sent|Accepted|Declined|Expired|Converte
d),
                    validUntil, subtotal, tax, total, approvalToken,
approvedByContactId, approvedAt;
                    QuoteLine  quoteId, type(Labor|Part|Expense|Text),
description, qty, unitPrice
Invoice             number, customerId, branchId, contractId?,
status(Draft|PendingApproval|Approved|Sent|PartiallyPaid|Paid|Overdue|
                    Void), issueDate, dueDate, periodStart, periodEnd,
currency, subtotal,
                    taxTotal, total, amountPaid, notes, version
InvoiceLine         invoiceId, workOrderId?, type, description, quantity,
unitPrice, taxCodeId, lineTotal
CreditNote          number, invoiceId, branchId, reason, amount, lines,
status(Draft|PendingApproval|Approved|Rejected)
Payment             invoiceId, amount, date,
method(Cash|BankTransfer|MobileMoney|Card|Cheque),
                    providerRef, branchId,
status(Pending|Approved|Rejected|Reversed), recordedById
TaxCode             name, rate, isInclusive, region
StatementRun        customerId, periodStart, periodEnd, openingBalance,
closingBalance, fileId
```

A work order priced for a customer produces billable lines from its labor (rate card), parts (price list) and expenses. An invoice never recalculates from live data; it stores its own line totals.

4.8 Notifications, audit, files and integrations

```
NotificationRule    event, channel(Email|SMS|WhatsApp|InApp), audience,
template, conditions, isActive
NotificationTemplate event, channel, locale, subject, body, isMandatory
Notification        recipientUserId? | recipientContactId?, channel, event,
status(Pending|Sent|
                    Delivered|Failed), providerRef, relatedType, relatedId,
retryCount, error
NotificationPref    userId|contactId, event, channel, enabled, quietHours
AuditLog            at, actorType(User|Contact|System|ApiKey), actorId,
action, entityType, entityId,
                    before (JSON), after (JSON), ip          // append only
File                storageKey, originalName, mime, sizeBytes, checksum,
uploadedById
WebhookEndpoint     url, secret, events[], isActive;  WebhookDelivery
endpointId, event, payload,
                    status, attempts, nextAttemptAt
ImportJob           entity, fileId, mapping, status, rowsOk, rowsFailed,
errorFileId
SavedView           userId, entity, filters, columns, sort, isShared
DashboardLayout     userId|role, widgets[]
ReportSchedule      reportKey, params, cron, recipients[], format
ApprovalRule        entity, action, condition (JSON: amount bands, branch,
department), levels[] (approverRoleId, scope rule), autoApproveBelow?,
fallbackAfterHours, isActive
ApprovalRequest     policyId, entityType, entityId, level, makerId,
status(Pending|Approved|Rejected|Withdrawn|Expired), checkerId?, decidedAt,
note, payloadHash   // DB CHECK (checkerId <> makerId)
OnCallShift         teamId, userId, start, end               (who is on
call, for Emergency orders)
Leave               userId, start, end, type                 (dispatch board
warnings)
ApprovalDelegation  fromUserId, toUserId, start, end   // never covers items
the delegate keyed
SyncAction          userId, deviceId, idempotencyKey (unique), entityType,
entityId, baseVersion,
                    payload, status(Applied|Conflict|Rejected),
resolvedById?, at   // offline outbox, section 7
ActivityLog         at, userId, sessionId, ip, deviceId, branchId, module,
action(Login|Logout|OtpFail|

View|Search|Export|Print|Download|Create|Update|Approve|Reject|...),
entityType,
                    entityId, meta (JSON)                  // append only,
section 9.6
```

5. Core workflows

5.1 Work request versus work order

A request captures a need. A work order is authorized, scheduled, assignable work. Keeping them apart lets anyone report a fault without being able to commit resources.

	Work request	Work order
Created by	Any requester, guest by QR, customer contact, email or WhatsApp, or staff on behalf of someone	Planner, supervisor, PM generator, or auto-converted from an approved request
Purpose	Describe the problem	Authorize and track the work
Status flow	Submitted, UnderReview, Approved or Rejected, Converted (or Cancelled)	Draft, Open, Assigned, InProgress, OnHold, Completed, Verified, Closed (or Cancelled)
Relationship	One request leads to zero or one order; several duplicate requests can attach to one order	Zero or one originating request
Auto rules	Duplicate detection by asset and open status, auto-approve for trusted requesters or categories	Auto-assign by team and skill, auto-close after verification

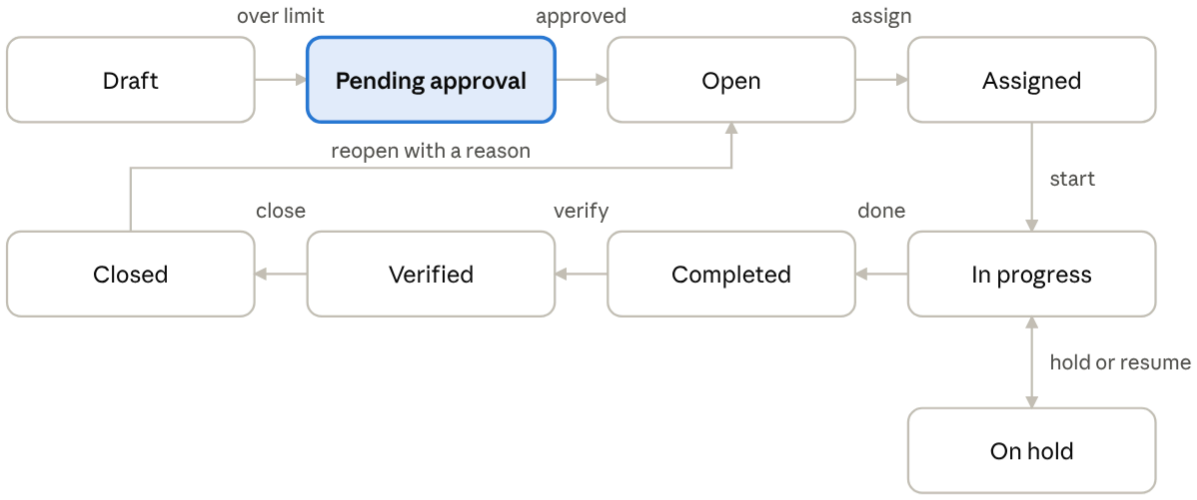
5.2 Work order lifecycle rules

Transition	Allowed by	Rule
Draft to Open	Planner, supervisor	Required: title, priority, asset or location. If the estimated cost is above the approval limit, the order waits in PendingApproval first (section 5.6)
Open to Assigned	Planner, supervisor, auto-assign	Needs a technician or a team; sets `slaRespondedAt` on first acknowledgement
Assigned to InProgress	Assigned technician	Sets `actualStart`; asset can move to UnderMaintenance

Transition	Allowed by	Rule
InProgress to OnHold	Technician, supervisor	Hold reason required (waiting parts, waiting customer, access, approval). Customer-caused reasons pause the SLA clock
OnHold to InProgress	Technician, supervisor	Resumes SLA clock; logs paused time
InProgress to Completed	Technician	All required tasks done; required photos and readings captured; failure, cause and remedy codes set for corrective orders
Completed to Verified	Supervisor or requester or customer contact	Only when `requiresVerification` is true; rejection returns the order to InProgress with a reason
Verified or Completed to Closed	Supervisor, system	Costs frozen, asset status restored, downtime closed, order becomes billable
Closed to Open (reopen)	Supervisor	Reason required; creates a linked history entry; reopens costs
Any open state to Cancelled	Supervisor	Reason required; reserved parts released

Every transition writes a status-history row and an audit entry in the same transaction as the change. Transitions that move money or stock, or exceed a cost limit, also pass through dual control (section 5.6).

A work order moves through states; high-cost orders wait for a second person



Pending approval: only when the estimated cost is above the approval limit; a different person approves.
Verified: only when the order requires verification by the requester or a supervisor.
Any open state can be cancelled with a reason; reserved parts are released.
On hold for a customer-caused reason pauses the SLA clock.

work order lifecycle · 9 states

The highlighted state appears only when an order's estimated cost is above the approval limit.

5.3 Emergency and out-of-hours

An Emergency work order skips approval, notifies the on-call technician and supervisor immediately by SMS and WhatsApp, starts the response clock at creation, and allows parts to be issued before a part request is approved (flagged for review). On-call rosters are a Team schedule: who is on call for each Team and period.

5.4 SLA engine

1. When an order is created, the system finds the SLA policy from the contract (contractor mode) or the company default (facility mode), then the rule for the order priority.
2. It computes `slaResponseDueAt` and `slaResolutionDueAt` using the business-hours calendar, so weekends and holidays do not count.
3. Customer-caused holds pause the clock; paused time is added back to both due times.
4. A job runs every 5 minutes. At 75 percent of the target it flags at-risk and notifies the assignee. At 100 percent it marks breached and escalates along the policy ladder (technician, supervisor, manager).

5. Reports show breach reasons and forecast breaches for the next 24 hours.

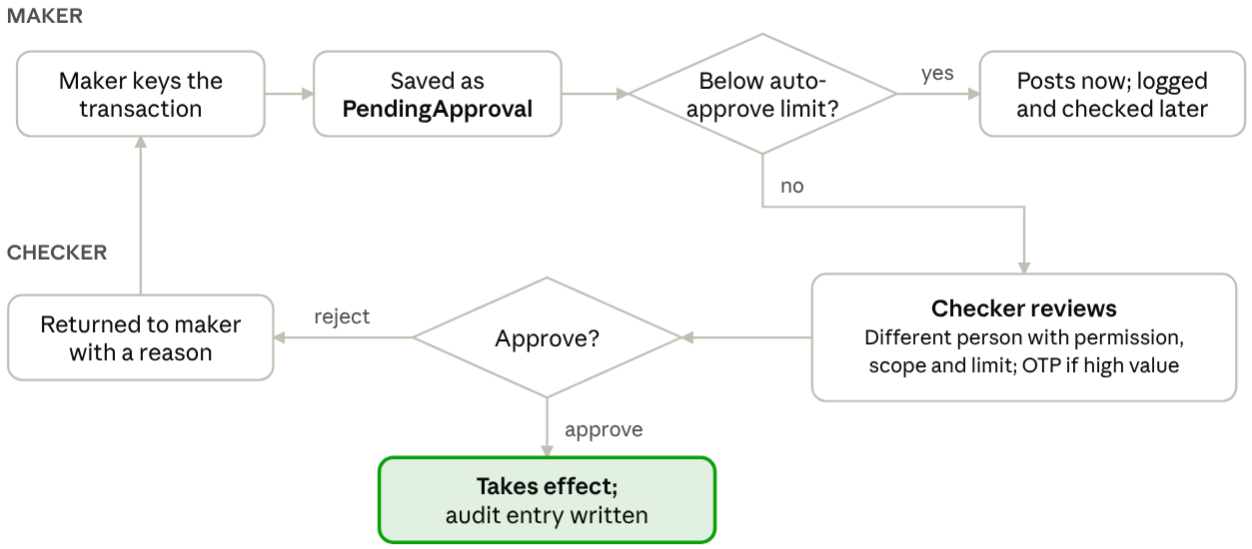
5.5 Preventive maintenance generation

1. Daily at the configured hour, the PM job selects active plans whose next due date falls within the lead time.
2. For each, it inserts a `PmOccurrence` first. The unique key (plan, due date) makes the job safe to re-run: a duplicate insert fails and is skipped.
3. It creates the work order from the occurrence, copies the checklist, reserves the parts kit, inherits priority, team and location from the asset, and assigns the default technician or team.
4. Time plans with anchor `FromCompletion` compute the next date from the actual completion of the last order; `FixedCalendar` plans keep their rhythm regardless.
5. Meter plans fire when a reading crosses the threshold since the last generation. Condition plans fire when a rule on readings is true (for example vibration above limit).
6. Missed occurrences (not started by due date plus grace) are marked Missed and counted in PM compliance.

5.6 Dual control (maker-checker)

Dual control is a system-wide rule and cannot be switched off: **a transaction keyed by one person takes effect only after a different, authorized person approves it.** Until then it is `PendingApproval` and has no effect on stock, balances, invoices or payments.

A second person must approve every controlled transaction



If the policy has more levels, these steps repeat with a different person at each level.
Auto-approval never applies to invoices, credit notes, payments or user and role changes.

dual-control approval flow · 2 roles, 2 decisions

A rejected item returns to the maker, who edits and resubmits; every round is logged.

Controlled transaction	Maker	Checker
Stock issue, return and transfer	Storekeeper or technician	Supervisor or branch manager
Stock adjustment, count variance	Storekeeper	Branch manager
Goods receipt	Storekeeper	Purchasing officer (not the buyer)
Purchase requisition and purchase order	Requester or buyer	Manager, tiered by amount
Vendor invoice match and vendor payment	Finance clerk	Finance manager
Quote release to customer	Planner or account manager	Manager above a set value
Customer invoice issue	Finance clerk	Finance manager
Credit note, write-off, refund, discount above limit	Finance clerk	Finance manager, plus a second level above a higher limit

Controlled transaction	Maker	Checker
Customer payment recording and reversal	Cashier	Finance manager
Contract, rate card and price list changes	Account manager	Manager
Work order cost overrun, cancel, reopen of a closed order	Supervisor	Branch manager
Asset retirement, disposal, value change	Asset owner	Branch manager
Users, roles, scope, approval policies, company settings	Admin	A second Admin

Rules the system enforces

1. **Maker cannot approve.** The service rejects it, the interface hides the approve action, and the database holds a `CHECK (checkerId <> makerId)` on the approval table, so a code defect cannot allow it.
2. **Last-editor rule.** If a checker edits the record, the checker becomes its maker and a different person must approve.
3. **Approval is bound to content.** The approval stores a hash of the record. Any change after approval cancels it and returns the record to `PendingApproval`.
4. **Checker must qualify.** The checker holds the checker permission for that transaction, has scope over the record's branch and department, and has an `approvalLimit` at or above the amount.
5. **Tiered levels.** Policies set amount bands. Example: up to the first limit, a supervisor; up to the second, the branch manager; above that, the branch manager and the finance manager. Each level is a different person, and nobody approves two levels.
6. **No bypass.** No role, including Admin, can approve its own entry or override the rule. If only the maker qualifies, the item escalates up the branch tree to the parent branch's approvers.
7. **Delegation is limited.** A checker can delegate to a qualified user for a period, never for items the delegate keyed.
8. **Rejection** returns the item to the maker with a reason. The maker edits and resubmits. Every round is logged.
9. **Emergency work.** Parts for an Emergency work order can be issued at once as provisional, then must be approved by a checker within 24 hours. Unapproved provisional items alert managers and block period close.

10. **Timers.** Pending approvals send a reminder at 50 percent of the target time and escalate at 100 percent.
11. **Step-up.** High-value approvals ask the checker for a fresh OTP.
12. **Approval inbox.** One queue on web and mobile showing the record, the maker, the amount and what changed. Email alerts carry a link; approval requires login with OTP, never approve-by-reply.

Operational settings for dual control

- **Auto-approve limits.** Per transaction type, branch and role, an admin can set a value below which an item is auto-approved and queued for after-the-fact review. Auto-approval never applies to invoices, credit notes, payments, user and role changes, or anything an Auditor has flagged. Every auto-approval is logged as such.
- **Alternate approver.** Each branch names an alternate approver. When the manager is the maker, or is absent, the item goes to the alternate; if there is none, it escalates to the parent branch's approvers. This prevents a single-manager outlet from stalling.
- **Policy snapshot.** A pending item keeps the approval policy that applied when it was submitted, so a policy change does not alter items already in the queue.
- **Offline actions.** An action queued offline is submitted for approval when it syncs, under the policy in force at sync time. If it no longer qualifies (for example the limit was lowered), the technician sees it as pending, not failed.

5.7 Notifications to the requester and others

The requester always knows where the request stands. Email is the default channel; SMS and WhatsApp are optional per company. Guests who reported by QR code receive email and a no-login tracking page (`/track/{token}`).

Event	Recipient	Content
Request received	Requester	Reference number, summary, tracking link
Request approved or rejected	Requester	Decision and reason
Converted to work order	Requester	Work order number, priority, expected date
Assigned, technician on the way	Requester	Technician name and arrival estimate
Work started	Requester	Start time

Event	Recipient	Content
On hold	Requester	Reason and what is needed (for example access or approval)
Completed, please verify	Requester	Summary, photos, verify or reopen links, reminder after 2 days
Closed	Requester	Completion summary and a one-tap satisfaction rating
Staff comment marked visible	Requester	The comment
SLA at risk or breached	Technician, supervisor, manager (customer on contracts)	Order and time left
Approval needed	Checker	Item, maker, amount, link
Approved or rejected	Maker	Decision and reason
PM due, low stock, overdue order	Assignee, storekeeper, supervisor	Item and due date
Quote sent, accepted, expiring	Customer, account manager	Quote and link
Invoice issued, reminder, overdue, payment received, receipt	Customer contact	Invoice, balance, pay link
OTP code, new-device login, password change	The user	Code or alert

Rules: every template is editable with merge fields and locale; mandatory events cannot be disabled; users choose channel and quiet hours for optional events; bursts are grouped into a digest; every message is logged with its delivery status and retried with back-off; email goes through a transactional provider with SPF and DKIM set.

6. Contractor mode

Contractor mode turns the shared core into a job-to-cash system: win the work, do it, prove it, bill it, collect the money.

6.1 The job-to-cash chain

1. **Enquiry.** A customer contact raises a request in the portal, or staff log one for a phone or email enquiry.
2. **Quote.** For work outside a contract, staff build a quote from the rate card and price list, optionally after a site visit work order. The customer receives an approval link (or approves in the portal). Approval converts the quote to a work order with the quoted lines as the budget.
3. **Job.** Work proceeds as in section 5. For contract work, the SLA policy and covered assets come from the contract.
4. **Proof.** The technician captures before and after photos, completes the checklist and collects the customer signature on the phone. The customer contact can verify in the portal.
5. **Billing.** Closing a billable order produces billable lines: labor hours times rate card, parts times price list or markup, expenses with markup, call-out fee. Finance reviews, groups orders into an invoice (one per job, or one per period per customer) and sends it.
6. **Collection.** Customer pays through a payment link or records offline payment. Overdue invoices trigger reminders. Statements summarize each customer account.

6.2 Contract and billing modes

Billing mode	How it bills	Typical client
Time and material	Actual labor, parts and expenses per job at contract rates	Hotels, estates, offices
Fixed fee	One recurring amount per period regardless of work, with included hours tracked	Clinics, schools
Retainer	Prepaid hours or amount drawn down by jobs; alert at 80 percent used	Small businesses
Per visit	Set price per scheduled PM visit	Lift, generator, air-conditioning servicing
Warranty	Zero billable, cost tracked against the warranty	Newly installed equipment

Contracts support renewal reminders, auto-renew, price escalation by percentage on anniversary, and coverage lists of assets and sites.

6.3 Customer portal

A separate route group and authentication realm. Contacts sign in with an email or phone one-time code, with no password. They never share a table, session or permission set with staff.

Feature	Detail
Dashboard	Open requests, jobs in progress, SLA status, outstanding balance
Requests	Create with photos, choose site and asset, see status and technician ETA
Jobs	Timeline of status, visible comments, photos, completion report and signature
Quotes	View, approve or decline with reason, download PDF
Invoices and payments	View, pay online, download invoice, credit notes and receipts
Statements	Account statement by period, aged balances, download PDF
Assets and history	Customer-owned assets with full service history and next due dates
Contracts	Active contracts, covered assets, SLA terms, included hours remaining
Users	A customer admin contact invites other contacts and sets which sites each can see

Portal rules: records resolve by UUID, never sequential id; every portal service method re-checks that the record belongs to the session's customer and, where limited, to the contact's allowed sites; internal comments and costs are never returned; quotes and invoices appear only after internal approval; all list endpoints paginate.

6.4 Customer-facing documents

Quotes, job completion reports, invoices, credit notes and statements are generated server-side as PDF from templates with the company's logo, legal name, tax number and footer text. Templates are editable by Admin, with merge fields. Each document is stored as a file and emailed or shared by secure link.

6.5 Contractor profitability

Because every labor hour, part and expense is costed and every billable line is priced, the system reports margin per job, per customer, per contract and per technician. This is the answer to the contractor's core question: which clients and which kinds of work actually make money.

6.6 Billing and invoicing in detail

Billing is a full module, not an export. Contractors bill customers; facility-mode companies use the same engine to charge maintenance cost back between departments and branches.

Invoice types

Type	Use
Job invoice	One closed billable work order
Consolidated invoice	Many orders for one customer and period, itemised
Contract invoice	Recurring fixed fee, retainer top-up or per-visit charge from a contract
Progress invoice	Milestones or percentage of a project or quote
Deposit or pro-forma	Advance payment requested before work starts
Credit note	Reduces or cancels an invoice; an issued invoice is never edited
Internal recharge	Facility mode: cost moved between departments or branches by approved journal

Invoice lifecycle: Draft, PendingApproval, Approved, Sent, PartiallyPaid, Paid, Overdue, Void (by credit note). The gapless invoice number is allocated at approval, so rejected drafts never leave a hole in the sequence. The maker who prepared the invoice cannot approve it.

Billing run

1. The unbilled work report lists closed billable orders not yet invoiced, by customer, contract, branch and period, with age.
2. Finance selects them, previews the invoices and generates drafts in a batch.
3. Drafts go to the approval queue (section 5.6). Approved invoices are issued and emailed with a payment link.

Pricing and tax

- Labor from the contract rate card by skill, with after-hours and overtime multipliers, call-out fee, minimum charge and travel time.
- Parts from the price list or cost plus markup percent; expenses with optional markup.
- Line and document discounts; any discount above the user's limit needs approval.
- Contract-covered work bills at zero while drawing down included hours; warranty work bills at zero and is costed to the warranty.
- Tax codes with inclusive or exclusive pricing, several rates on one invoice (for countries that charge VAT plus separate levies), withholding tax, and a rounding rule per company.

Collections

- Payment terms per customer; payment links for card, mobile money and bank transfer; offline payments recorded by the cashier and approved by finance.
- Payments allocate to the oldest invoice first or to a chosen invoice; part payments, deposits and credit on account are supported; over-payments become credit.
- Receipt emailed on every approved payment.
- Reminder schedule: before due date, on due date, then at 7, 14 and 30 days, escalating to the account manager. A dispute flag pauses reminders.
- Credit limit and credit hold: new jobs for a customer over the limit need approval to proceed.
- Bank statement import (CSV) suggests matches for reconciliation.

Statements and reporting

- Monthly customer statements with ageing buckets (current, 30, 60, 90, over 90 days), emailed automatically and available in the portal.
- Reports: unbilled work, aged debtors, revenue by customer, contract, branch and department, margin per job, days sales outstanding, collection rate, invoice approval turnaround.
- Accounting hand-off: journal export with GL account mapping per income and cost category. Branch and department are carried as dimensions so the accounting system can report by cost centre.

7. Mobile, offline, QR and sensors

7.1 Technician PWA

The technician app is an installable progressive web app on the same codebase, so there is no app store dependency and one team builds it. It is designed for a phone held in one hand on a site with weak signal.

- **My day:** today's jobs sorted by time and priority, with map links to the site and one-tap call to the requester.
- **Job screen:** start, hold and complete with large buttons; checklist, readings, photos, parts, labor timer, comments, signature.
- **Offline:** assigned jobs, asset details, checklists and the parts catalogue are cached. Actions queue in an IndexedDB outbox and sync when online. Each queued action carries the record `version` and an idempotency key.
- **Conflict handling:** if the server version has moved, the app shows both versions of the changed fields and lets the technician keep theirs, keep the server's, or merge. Status transitions that are no longer valid are rejected with an explanation.
- **Scanning:** the camera reads QR codes and barcodes to open an asset, issue a part or confirm a location.
- **Low bandwidth mode:** photos are compressed on the device before upload; lists load in small pages.
- **Push notifications:** new assignment, urgent job, SLA at risk, approval needed.

Offline and dual control. Actions that need approval (part issues above the limit, cost overruns) are queued offline as requests, never as completed transactions. Stock and balances change only after the server records the approval, so an offline device can never move stock by itself.

7.2 QR-based reporting

Every asset and every location has a printable QR label. Scanning opens a lightweight page: the item name, a short fault form with photo, and the requester's name and phone (no account needed for guests, rate-limited and captcha protected). A hotel room, a hospital bed or a generator each become a one-scan fault report. Admin prints label sheets in bulk from the asset list.

7.3 Dispatch board and calendar

Within their branch and department scope, the planner sees a board of unassigned orders and a calendar by technician and Team. Drag assigns or reschedules, warns on skill mismatch, overlapping jobs and technician leave, and shows travel distance when site

coordinates exist. Contractor mode adds route ordering of the day's jobs.

7.4 Sensors and condition monitoring (R3)

An inbound endpoint accepts readings from gateways, building systems or manual spreadsheets (`POST /api/v1/readings` with an API key and asset tag). Readings feed meter plans and condition rules, and can open a request automatically when a threshold or trend is crossed. No device management is built; ZemiFix consumes readings and does not run the sensor network.

7.5 Offline synchronization test cases

The sync layer is built and tested as its own step, against these cases, before the technician app is released.

Case	Expected result
Two technicians edit the same order offline, one syncs first	Second sync shows a conflict screen with both versions of each changed field
Technician completes an order that a supervisor cancelled meanwhile	Sync rejected with a clear message; work done is kept as a note, not lost
The same queued action is sent twice (network retry)	Applied once, thanks to the idempotency key
Part issue queued offline, stock reaches zero before sync	Request stays pending; technician is told stock is short
Approval policy or limit changes while the device is offline	Action is judged under the policy at sync time (section 5.6)
Device offline for several days	Queue syncs in order; expired sessions require OTP before upload
Photo upload interrupted	Resumes from where it stopped; record shows the photo as pending
Clock on the device is wrong	Server time is used for audit; the device time is stored as a note
User deactivated while offline	Sync rejected, device data wiped on next contact
Device lost	Admin revokes the session; cached data is encrypted and cleared on next contact

8. Inventory, purchasing and analytics

8.1 Inventory rules

- **Ledger first.** `StockTransaction` is append-only and is the audit truth. `Stock.quantityOnHand` is the fast read, updated in the same transaction. A nightly job asserts the two agree and alerts on drift.
- **Concurrency.** Issuing stock runs in one transaction that locks the stock row (`SELECT ... FOR UPDATE`), checks availability, decrements, and writes the ledger row. Two simultaneous issues cannot drive stock below zero unless the company allows negative stock.
- **Reservation.** Parts on a planned work order or PM kit are reserved, lowering available stock without moving it.
- **Stores and bins.** Several stores (main store, van stock, site cupboard) with bins. Transfers move stock between stores through an in-transit state. Technician van stock is a store assigned to a user.
- **Costing.** Weighted average cost by default, FIFO optional. The cost of a part on a work order is the cost at the moment of issue.
- **Counts.** Cycle counts by store or bin produce variance lines that need approval before an adjustment is posted.
- **Traceability.** Serial and batch tracked parts record which asset and work order consumed them.

Approval without slowing repairs. Dual control applies to stock, but a technician should never wait for a filter. Each branch sets an auto-approve limit per part category and per work order (value and quantity). Issues below the limit post at once and are logged as auto-approved; a checker reviews them later in a batch. Issues above the limit, and all adjustments, transfers and write-offs, wait for a checker.

8.2 Purchasing flow

Requisition (from a part request, low stock alert or work order need), approval by configured rule, purchase order to vendor by email or PDF, goods receipt against the order (partial allowed), vendor invoice captured and matched three ways (order, receipt, invoice), then marked paid. Receiving posts a stock receipt and updates part cost. Vendor score combines on-time delivery, price variance and rejection rate.

8.3 Key performance indicators

Every KPI has one definition, computed from the work order and downtime data, and is filterable by date range, asset category, location, Team, priority and customer.

KPI	Definition	Why it matters
MTTR	Total repair time of corrective orders divided by their count	How fast failures are fixed
MTBF	Operating time divided by number of failures per asset	How reliable equipment is
Availability	(Scheduled time minus downtime) divided by scheduled time	Uptime the business feels
PM compliance	PM orders completed on time divided by PM orders due	Whether prevention is happening
Planned vs reactive ratio	Preventive hours divided by total hours	Maturity of the program
Backlog age	Count and age of open orders by priority	Workload health
SLA attainment	Orders within SLA divided by orders with an SLA	Contract performance
First-time fix rate	Corrective orders closed without a reopen or repeat in 30 days	Quality of work
Technician utilization	Logged labor hours divided by available hours	Capacity use
Cost per asset	Labor, parts and expenses per asset, per period, with lifecycle total	Repair-or-replace decisions
Maintenance cost vs asset value	Annual cost divided by purchase cost	Flags money pits
Stock turnover and stockouts	Issues divided by average stock, count of stockouts	Inventory efficiency
Margin per job (contractor)	Billed amount minus labor, part and expense cost	Profitability
Days sales outstanding (contractor)	Receivables divided by daily billing	Cash collection

8.4 Reports and dashboards

- **Role dashboards:** manager (backlog, SLA, PM compliance, cost), technician (my day), storekeeper (low stock, pending requests), finance (aged debtors, unbilled work), executive (uptime, cost trend, top failing assets).
- **Report library:** work order summary, backlog, overdue, downtime by asset, failure analysis (Pareto of failure codes), cost by asset and category, labor by technician, PM schedule and compliance, inventory valuation, purchase history, customer statement, aged receivables, revenue by customer and contract, margin per job.
- **Tools:** saved filters, shareable views, CSV and Excel export, PDF export, scheduled email delivery, and a read-only SQL reporting view layer so a BI tool can connect later.

8.5 Additional features that make the product complete

Feature	What it does
Maintenance budgets	Annual budget per branch, department and asset category; actual versus budget; alerts at 80 and 100 percent; spend over budget needs approval
Compliance and inspections	Statutory inspections, certificates and calibration schedules (fire equipment, lifts, generators, medical devices) with expiry alerts and an audit pack export
Permit to work and safety	Risk level, safety checklist and lock-out steps on high-risk jobs, with supervisor sign-off before start
Subcontractors	Assign a work order to an external vendor, send a link for quote and completion, match the vendor invoice to the order
Warranty management	Track warranty by asset, flag work that should be claimed, record claim outcome
Timesheets	Technician time capture with GPS check-in at site, overtime rules, and timesheet approval by a different person
Asset lifecycle	Depreciation, replace-versus-repair view from cost history, disposal workflow under dual control
Customer satisfaction	One-tap rating on closure, comments, technician and branch scores in reports

Feature	What it does
Knowledge base	Fixes recorded against failure codes, manuals and troubleshooting steps suggested on similar jobs
Recurring service agreements	Visit schedules for contract customers with automatic job creation and billing
Map and command view	Live map of jobs and technicians for dispatchers
Document library and e-signature	Manuals, method statements and signed completion reports kept against the asset and job
Data tools	Spreadsheet import and export, bulk edit with approval, archive of closed records

9. Security, integrations and non-functional requirements

9.1 Identity and access

Topic	Decision
Staff login	Email and password (argon2id), then a mandatory one-time code (OTP) by email or SMS; authenticator app as an alternative; SSO through OIDC in R2. Detail in 9.7
Portal login	Email or phone one-time code only, short session, no remember-me
Sessions	Server-side, revocable, 8-hour lifetime, 2-hour idle timeout, list and revoke from profile
Permissions	Role-based with granular resource and action pairs, checked in services. Hiding a button is presentation only
Custom roles	Admin can clone a system role and adjust permissions
Rate limits	Login, one-time code request and verify, public QR form, API keys, all backed by Postgres counters
Files	Type and size validation, malware scan status, stored outside the web root, served by short-lived signed URL

Topic	Decision
Audit	Append-only, tamper-evident log of every user activity, including logins, views, exports and approvals. Detail in 9.6
Secrets	Environment variables or secret manager; webhook secrets and API keys stored hashed or encrypted
Transport and data	TLS everywhere, encrypted backups, encryption at rest through the database host
Privacy	Personal data export and erasure tooling, retention settings, consent record for SMS and WhatsApp. Designed to satisfy Ghana's Data Protection Act, 2012 (Act 843) and GDPR-style requirements

Authorization has three layers, all checked in the service layer and backed by database Row-Level Security on branch scope: the **role** (what actions are allowed), the **access scope** (which branches and departments the user can reach), and the **dual-control rule** (who may approve what). Customer contacts sit outside the branch model; their reach is their customer and permitted sites, re-checked in every portal service method.

9.2 Role matrix (system defaults)

Role	Requests	Work orders	Assets	Inventory and purchasing	Billing	Settings
Requester	Create, view own	View own requests' orders	View	None	None	None
Technician	View	View assigned, update, log labor and parts	View, record readings	Request parts	None	None
Planner	Triage, convert	Full, assign, schedule	Full	View, request	None	None
Supervisor	Approve, reject, convert	Full, verify	Full	Approve requests	View	None
Storekeeper	View	View, issue parts	View	Full stock, receive	None	None
Finance	View	View costs	View	Purchasing approve, vendor invoices	Full	None
Manager	Full	Full	Full	Full	View, approve	Limited

Role	Requests	Work orders	Assets	Inventory and purchasing	Billing	Settings
Admin	Full	Full	Full	Full	Full	Full
Customer contact	Create, view own customer's	View own customer's, comment, verify	View own customer's	None	View, pay	None

Access scope. Set per user and combined with the role.

Scope	The user can see and act on
Own	Records the user created or is assigned to
Department	Records of the user's branch in the user's department or its sub-departments, plus unassigned records for triage
Branch	All records of the user's branch, every department
Branch and below	The user's branch and all its child branches
Company	Everything

Extra reach can be granted for a period through a scope grant (for example a regional supervisor covering two branches). Roles hold maker and checker rights as separate permissions, so a user can be allowed to create invoices without being allowed to approve them.

Additional system roles. Auditor: read-only access to the audit trail, approval history and reports, with no right to change anything. Cashier: records customer payments (maker only; finance approves). Purchasing officer: raises and approves purchase orders and goods receipts within limits. Account manager (contractor mode): owns customers, contracts and quotes. Roles can be cloned and adjusted by Admin, and every role change needs a second Admin to approve.

9.3 Integrations and API

Integration	Approach	Release
Public REST API	Versioned (`/api/v1`), OpenAPI document generated from Zod schemas, API keys with scopes, pagination `{ data, meta }`, `Idempotency-Key` on writes	R2

Integration	Approach	Release
Webhooks	Signed events for requests, work orders, invoices, payments, stock; retries with back-off and delivery log	R2
Email	Transactional provider through SMTP or API; inbound email creates requests (R2)	R1, R2
SMS	Provider interface; example adapters for Arkesel and Africa's Talking in Ghana and the region, Twilio elsewhere	R1
WhatsApp	Business API provider interface for notifications and request intake	R2
Payments	Provider interface: Paystack or Flutterwave (cards and mobile money, including MTN MoMo in Ghana), Stripe elsewhere; webhooks verified by signature	R1, R2
Accounting	CSV and journal export first; connectors for QuickBooks, Xero and Zoho Books later	R2, R3
Calendar	ICS feeds for a technician's schedule; Google and Microsoft calendar sync	R2, R3
Import	Spreadsheet import for assets, parts, customers, locations with preview, mapping and error file	R1
SSO and directory	OIDC (Microsoft Entra, Google Workspace)	R2

API conventions

- Base path `/api/v1`, JSON only. The web app uses its session cookie; integrations use API keys (bearer token) with scopes. An API key acts as a service identity: it can be a maker, never a checker, so no integration can approve a transaction.
- Writes that need approval return `202 Accepted` with the approval id and `PendingApproval` state; they never return success as if the change had taken effect.
- Lists paginate as `{ data, meta: { total, page, perPage } }`, with cursor pagination for large sets. Filtering and sorting use `?filter[status]=Open&sort=-createdAt`.
- Errors use the problem-details format with a stable `code`: `400` invalid request, `401` not signed in, `403` permission or scope, `404` not found or outside scope, `409` version conflict, `422` validation, `429` rate limit.

- Every write accepts an `Idempotency-Key`. Rate-limit headers are returned on every response. The OpenAPI document is generated from the Zod schemas and published per version.

Area	Example endpoints
Requests	`POST /work-requests`, `GET /work-requests`, `POST /work-requests/{id}/approve`
Work orders	`GET /work-orders`, `POST /work-orders/{id}/assign`, `/status`, `/tasks`, `/labor`, `/parts`, `/comments`, `/attachments`
Assets	`GET /assets`, `POST /assets/{id}/meter-readings`, `GET /assets/{id}/history`
Preventive maintenance	`GET /pm-plans`, `GET /pm-occurrences`
Inventory	`GET /parts`, `GET /stock`, `POST /part-requests`, `GET /stock-transactions`
Customers and billing	`GET /customers`, `POST /quotes`, `POST /invoices`, `POST /invoices/{id}/payments`
Approvals	`GET /approvals?status=Pending`, `POST /approvals/{id}/approve`, `POST /approvals/{id}/reject` (user sessions only)
Audit	`GET /audit-log` (Auditor scope only)
Sensor readings	`POST /readings`

9.4 Internationalization

- All text through message catalogs; English at launch, French and Portuguese next for Africa-wide use.
- Currency, tax codes and number and date formats are company settings. Tax lines support inclusive and exclusive pricing, several rates per invoice, and withholding where required.
- Invoices can be issued in a document currency different from the base currency in R3, with a stored exchange rate per document.
- Right-to-left layout is not in scope.

9.5 Non-functional targets

Area	Target
Availability	99.5 percent monthly on the default single-VPS deployment; 99.9 percent on the managed option
Performance	List pages under 500 ms and writes under 800 ms at the 95th percentile, with 100,000 assets and 1 million work orders
Scale	500 concurrent staff users and 200 concurrent portal users on one web container
Recovery	RPO 15 minutes with WAL archiving (24 hours without); RTO 4 hours; monthly restore test
Accessibility	WCAG 2.1 AA for staff app and portal
Browser support	Last two versions of Chrome, Edge, Safari, Firefox; Android Chrome and iOS Safari for the PWA
Data growth	Partition `AuditLog`, `StockTransaction`, `Notification` and `MeterReading` by month when over 10 million rows
Testing	Rules covered by integration tests on real Postgres; end-to-end tests for the 10 critical journeys; load test before each major release
Observability	Request tracing, job dashboards, error alerts, uptime check, SLA-job heartbeat

9.6 Audit trail: every user activity

The audit trail answers who did what, when, from where, and what changed, for every user, role and API key.

Aspect	Design
What is logged	Logins, logouts and failed OTPs; every create, update, status change and delete with before and after values; approvals and rejections with reason; role, permission and scope changes; views of sensitive records (invoices, payments, customer and user records); searches; exports, prints and downloads; settings changes; API calls

Aspect	Design
Context on each row	UTC time, user, role, session, IP address, device, branch, module, action, record, correlation id, reason where required
Tamper evidence	Append-only tables; the application database account has insert and read rights only; each row stores a hash of the previous row, and a daily anchor hash is written to separate storage
Access	An Auditor role views, filters and exports the trail but cannot change anything. Viewing the trail is itself logged
Retention	Configurable; default 7 years for financial records, 2 years for view and search events, archived to cheap storage after 12 months
Alerts	Repeated failed OTPs, approvals outside working hours, bulk exports, a user approving many items from the same maker, access to records outside the user's usual branch
Reports	User activity, approval history by user, maker-checker pairing report (to expose collusion), record history, login history

Sizing (planning assumption, to be validated). Take 200 active users generating about 400 logged events a day each: about 80,000 rows a day, roughly 29 million a year. At about 600 bytes a row that is about 17 GB a year before indexes. The log is partitioned by month; view and search events move to cold storage after 12 months while financial events stay for the full retention period; auditor queries use indexes on user, record and time.

9.7 OTP login flow

1. The user enters email and password. A correct password creates a pending session, not a logged-in one.
2. The system sends a six-digit code, valid for 5 minutes and usable once, stored only as a hash, by email (default) or SMS.
3. The user enters the code. After 5 wrong attempts the account locks for 15 minutes; resending is limited to 3 per 10 minutes.
4. On success the session is marked OTP-verified and the sign-in is logged. A new-device sign-in sends a security alert to the user.

5. Optional trusted device (up to 30 days) can be allowed by Admin, and is always off for approvers and finance roles.
6. **Step-up OTP:** a fresh code is needed to approve high-value items, change roles or bank details, and export data.
7. **Recovery:** password and OTP resets for staff are started by an Admin and approved by a second Admin (dual control). Admins never recover by email alone.
8. **Portal:** contacts sign in with email or phone and OTP only, with no password.
9. An authenticator app (TOTP) is available as a fallback where email or SMS delivery is unreliable.

OTP delivery and cost. Email is the default channel and costs almost nothing, but codes must arrive within seconds: use a transactional provider on a dedicated sending domain with SPF, DKIM and DMARC, and watch the bounce rate. SMS has a per-message cost that varies by provider and network. Planning formula: monthly SMS cost = users x logins per day x working days x share sent by SMS x price per message. Controls: email by default, SMS for approvers and as a fallback, a daily SMS budget with an alert, a cap per user per day, delivery rate tracked per network, an offer of the other channel after two failed deliveries, and the authenticator app as a no-cost alternative.

10. Build order, releases and decisions

10.1 Release plan

Release	Goal	Contents
R1: Core	A hotel, clinic or contractor can run daily maintenance and bill for it	Company, branch tree and department tree with scope enforcement, OTP login, dual-control engine, activity audit trail, requester email notifications, core entities, requests, work orders, PM (time and meter), assets and QR, technician PWA with offline, inventory with ledger, purchase orders, customers, contracts and SLA, portal (requests, jobs, invoices), quotes, invoicing and payments, notifications (email, SMS), role dashboards, core reports, import, audit

Release	Goal	Contents
R2: Competitive	Match or beat the category on depth	Tiered multi-level approval policies and delegation, requisitions and three-way match, stores and transfers, van stock, retainers and recurring agreements, statements, credit notes, WhatsApp, inbound email intake, public API and webhooks, SSO, report scheduling, dispatch board with skills, parts kits, bill of materials, accounting export
R3: Advantage	Differentiate	Condition monitoring and sensor ingestion, failure-code analytics and predictive PM suggestions, AI triage of requests, natural-language search, multi-currency documents, calendar sync, accounting connectors

10.2 R1 build order

Feed one step at a time to the AI coding tool, referencing this document, and review each before moving on.

1. **Foundation.** Repository, CI, Docker Compose, Prisma schema for company, branch tree, department tree, users, roles, permissions and scope, auth with mandatory OTP, activity audit trail, dual-control engine, event bus, number sequences, file storage, queue and worker, seed data.
2. **Conventions proof.** Build the first vertical slice (Locations and Assets) completely: schema, service, actions, pages, tests, audit, optimistic concurrency. Freeze the conventions here; every later module copies this slice.
3. **Work requests** with QR intake and guest reporting.
4. **Work orders:** lifecycle, status history, tasks, labor, parts, expenses, comments, attachments, downtime, failure codes.
5. **Preventive maintenance:** templates, plans, idempotent generation, meter readings.
6. **Inventory:** stores, parts, ledger, locked issuing, part requests.
7. **Purchasing:** vendors, orders, receiving.
8. **Technician PWA:** offline cache, outbox, sync, conflict screen, scanning.

9. **Customers, contracts and SLA:** sites, contacts, rate cards, SLA engine and breach job.
10. **Portal and quotes:** portal realm, requests, jobs, quote approval link.
11. **Invoicing and payments:** billable lines, invoices, PDF, payment provider adapter and webhook, overdue job.
12. **Notifications:** templates, rules, queue dispatch, preferences, delivery webhooks.
13. **Reports and dashboards:** KPIs, role dashboards, exports.
14. **Import, hardening and release:** spreadsheet import, security review, load test, backup and restore drill.

Effort estimate (to be confirmed by the tech lead): with a lead and two developers using AI coding tools, R1 is roughly 6 to 8 months at the full scope in this document. Shipping it as one release is accepted, but building it in two internal milestones (core operations and controls first, then contractor billing, portal and the offline app) keeps risk visible; R2 a further 3 to 4 months.

10.3 Risks

Risk	Mitigation
Scope creep: this is a large product	Hold R1 contents fixed; everything else waits for R2 or R3
Offline sync defects	Build sync as its own step with a test harness for conflicts; limit offline writes to technician actions
Billing errors damage trust	Invoices store their own totals; money always Decimal; every billing rule covered by tests
Portal data leak across customers	Authorization re-check in every portal service method plus automated tests that one customer cannot read another's records
AI coding tool drift in conventions	The second build step freezes the pattern; review each step against it
Single VPS outage	Daily restore-tested backups, documented rebuild runbook, managed-hosting upgrade path

10.4 Decisions to confirm

1. **One company per deployment.** Selling ZemiFix to many businesses means one deployment per customer, automated as in section 3.6. Confirm.

2. **Approval limits.** Default amount bands and auto-approve limits per transaction type, branch and role, agreed at kickoff with each customer.
3. **Alternate approvers.** Every branch with a single manager needs a named alternate (section 5.6).
4. **OTP channel and SMS budget.** Email only at launch, or email plus SMS with a budget (section 9.7).
5. **Hosting default.** Docker on a VPS versus a managed platform.
6. **First payment adapter** for the launch market.
7. **Pricing and licensing model** (per user, per asset, or per deployment), which drives the licence and usage records in section 3.6.
8. **Competitor validation.** A dated, sourced comparison with three to five competitors as the first R1 task (section 2).
9. **Pilot clients for R1.** One facility-mode and one contractor-mode client, with their requirements kept out of this document.
10. **Effort estimate.** The tech lead confirms the 6 to 8 months for R1 before commitments are made.

10.5 Data migration and onboarding

A CMMS is only as good as the data loaded into it. Each customer follows the same seven phases.

Phase	Activities	Output
1. Discover	Collect the branch and department structure, user list, asset registers, part lists, customers and contracts; agree approval limits and alternate approvers	Signed-off scope and approval matrix
2. Prepare	Spreadsheet templates per entity with validation rules (unique asset tags, required fields, parent exists) and a worked sample file	Clean files ready to import
3. Import	Dry run with preview, column mapping, error file, and rollback by import batch. Order: branches, departments, users, locations, categories, assets, parts, vendors, customers, contracts, PM plans, open work orders	Loaded data, zero unresolved errors

Phase	Activities	Output
4. Opening balances	Opening stock and opening receivables loaded as controlled transactions, so a second person approves each batch	Approved opening balances
5. Configure	Roles, scopes, approval policies, SLAs, tax codes, numbering, invoice and notification templates, QR labels printed and applied	Working configuration
6. Train	Role-based sessions (requester 15 minutes, technician 1 hour, approver, finance), quick guides, a practice environment	Trained users
7. Go live	Pilot at one branch, parallel run, then roll out branch by branch, with two weeks of hypercare support	Live company

Success measures reported at 30 and 90 days: share of requests raised in the system, PM compliance, time to close, approval turnaround, and active users.

11. User interface specification

11.1 Principles

- Requester and technician screens are designed for a phone first; planner, finance and manager screens are designed for dense desktop work.
- The same record looks the same everywhere: a header with number, status and key facts, then tabs.
- Every list has saved views, column choice, search and bulk actions. Every action a user may not take is hidden, and the server still enforces it.
- Plain language, no jargon, status words that match this document.
- Accessible (WCAG 2.1 AA), keyboard friendly, dark mode, company logo and accent colour, print views for orders, invoices and QR labels.

11.2 Navigation

A left menu grouped by module (Work, Assets, Maintenance plans, Inventory, Purchasing, Customers and billing, Reports, Settings), a top bar with global search, a Create button, the approval inbox with a count, notifications, a branch selector for users whose scope

covers several branches, and the profile menu. Menu items appear only for permissions the user holds.

11.3 Key screens

Screen	Purpose	Main elements
Role dashboard	See what needs attention now	Cards for backlog, SLA at risk, pending approvals, PM due, low stock, cost trend, tailored to the role
Quick request	Report a fault in 30 seconds	Asset or room (scan or search), description, photo, priority; confirmation with tracking link
Work order detail	Run one job	Header (number, status, priority, SLA clock); tabs for Tasks, Parts, Labor, Costs, Comments, Attachments, Approvals, History and Audit
Dispatch board and calendar	Plan and assign	Unassigned queue, technician lanes, drag to assign, skill and leave warnings, map toggle
Asset detail	Know one asset	Hierarchy, meters, downtime, BOM, documents, PM plans, cost history, QR label
Approval inbox	Decide quickly and safely	Item, maker, amount, what changed, approve or reject with reason, OTP step-up for large amounts
Inventory	Keep stock right	Stock by store and bin, requests, issues, counts, reorder suggestions
Invoicing	Bill customers	Unbilled work, billing run, invoice editor, approval status, payments, statements
Customer portal	Customers self-serve	Dashboard, requests, jobs, quotes, invoices, statements, assets
Audit viewer	Investigate	Filter by user, record, action, branch and date; export; maker-checker pairing report

Screen	Purpose	Main elements
Administration	Configure	Company, branch tree, department tree, users, roles, scopes, approval policies, templates, numbering, integrations